

Hotel Room Booking Management System

Task 1 - Create PostgreSQL database.

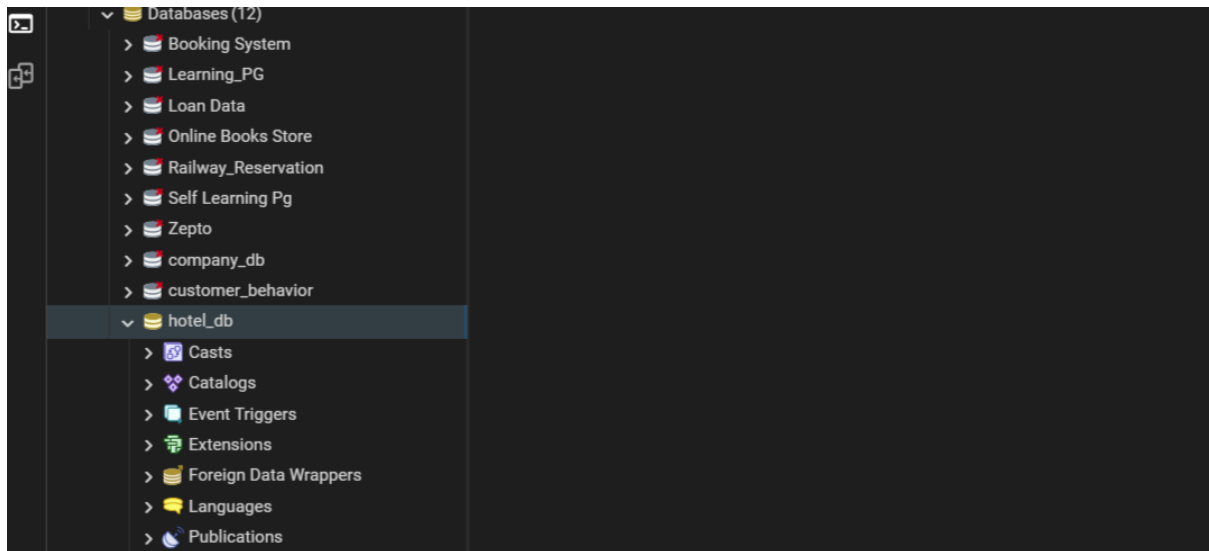


Figure 1: Database Created

Task 2 - Create database.py, models.py, schemas.py and main.py Files

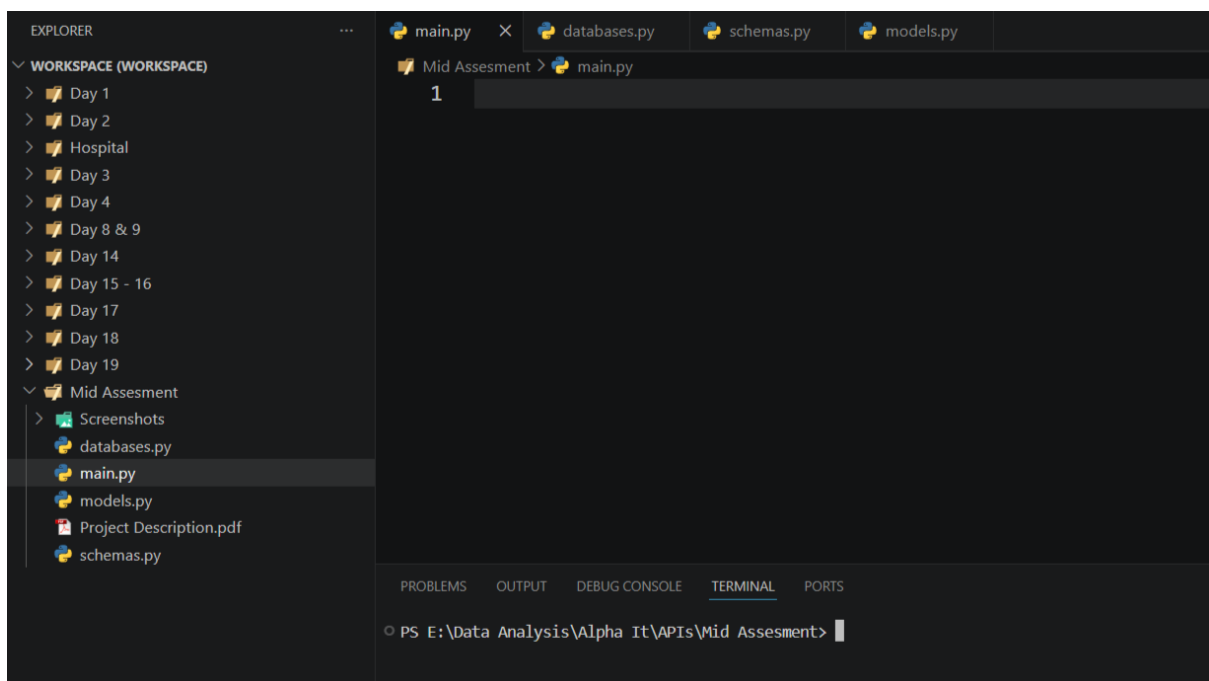
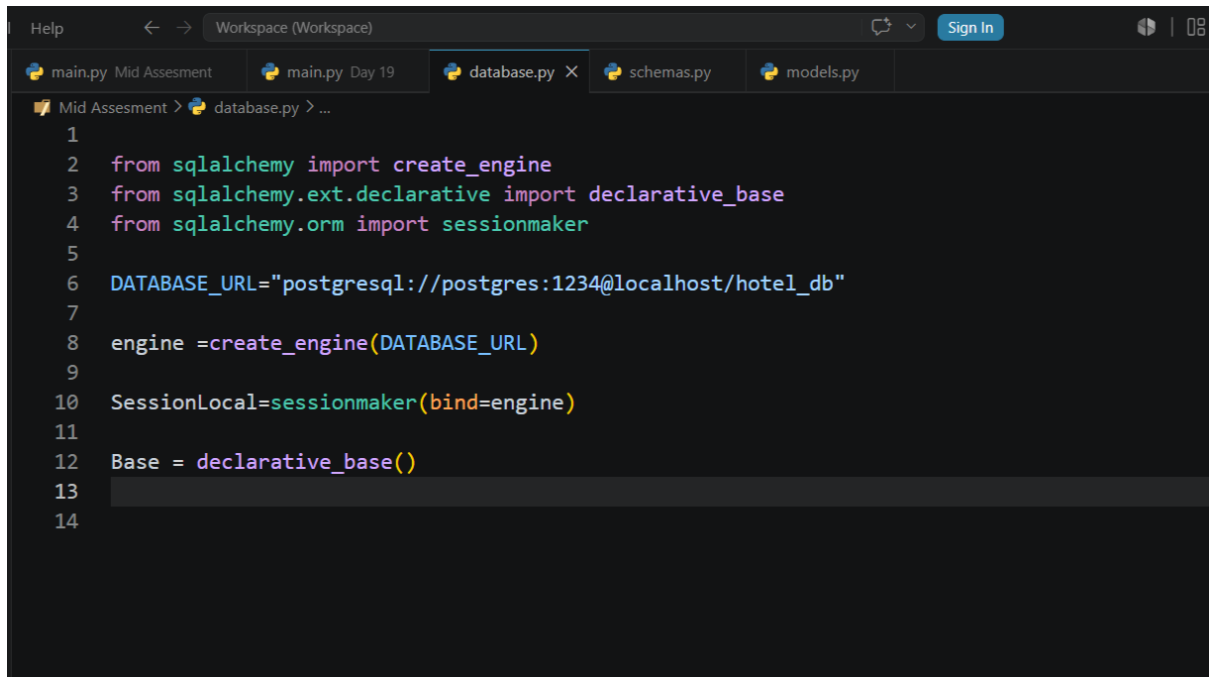


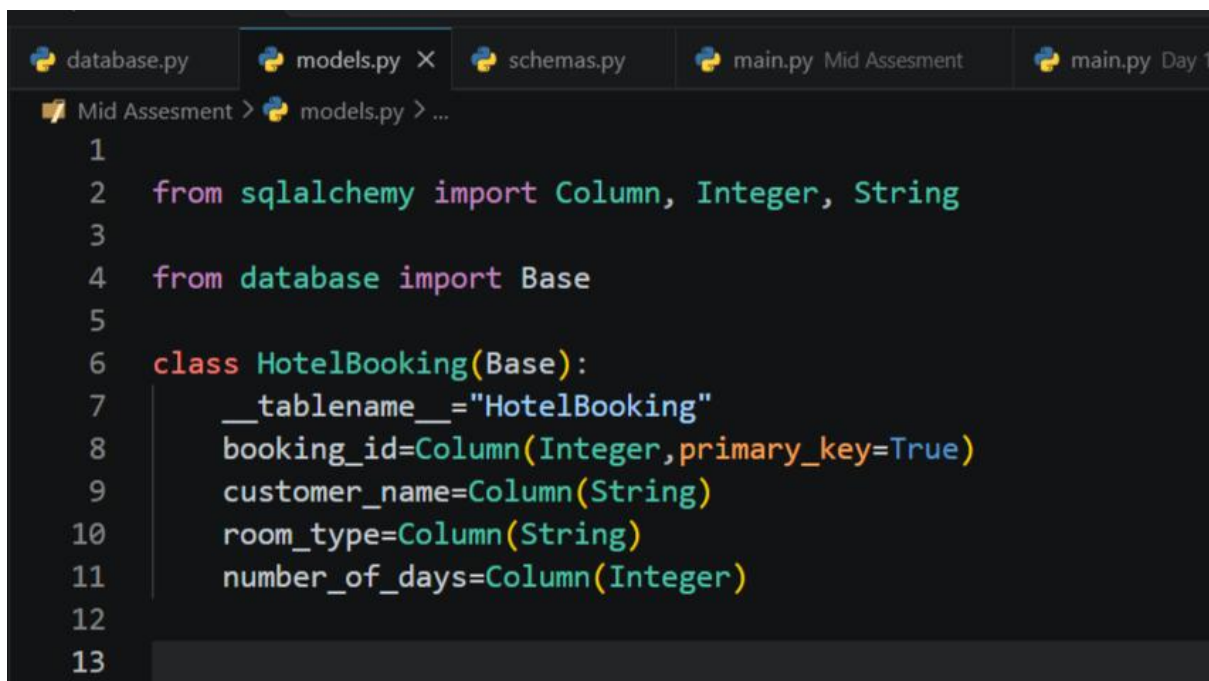
Figure 2: Required Files Created

Task 3 - Create SQLAlchemy model



```
1
2 from sqlalchemy import create_engine
3 from sqlalchemy.ext.declarative import declarative_base
4 from sqlalchemy.orm import sessionmaker
5
6 DATABASE_URL="postgresql://postgres:1234@localhost/hotel_db"
7
8 engine =create_engine(DATABASE_URL)
9
10 SessionLocal=sessionmaker(bind=engine)
11
12 Base = declarative_base()
13
14
```

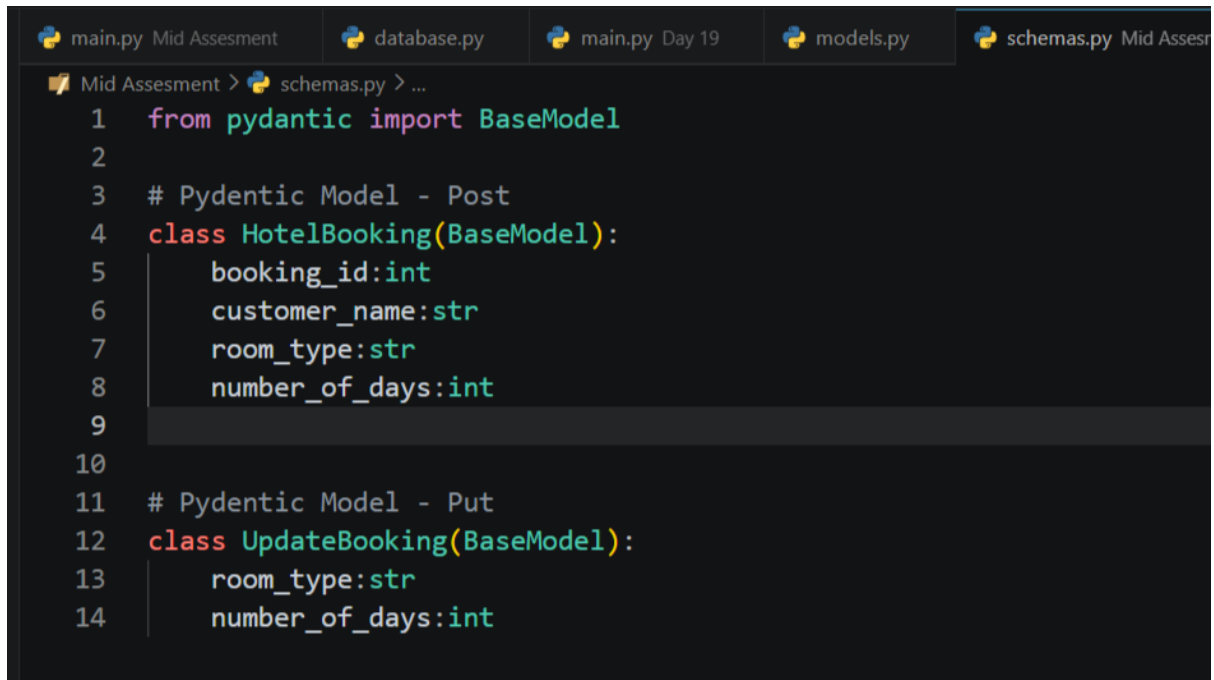
Figure 3(a): SQLAlchemy model Created



```
1
2 from sqlalchemy import Column, Integer, String
3
4 from database import Base
5
6 class HotelBooking(Base):
7     __tablename__="HotelBooking"
8     booking_id=Column(Integer,primary_key=True)
9     customer_name=Column(String)
10    room_type=Column(String)
11    number_of_days=Column(Integer)
12
13
```

Figure 3(b): Table Created Using Python

Task 4 - Create Pydantic Model

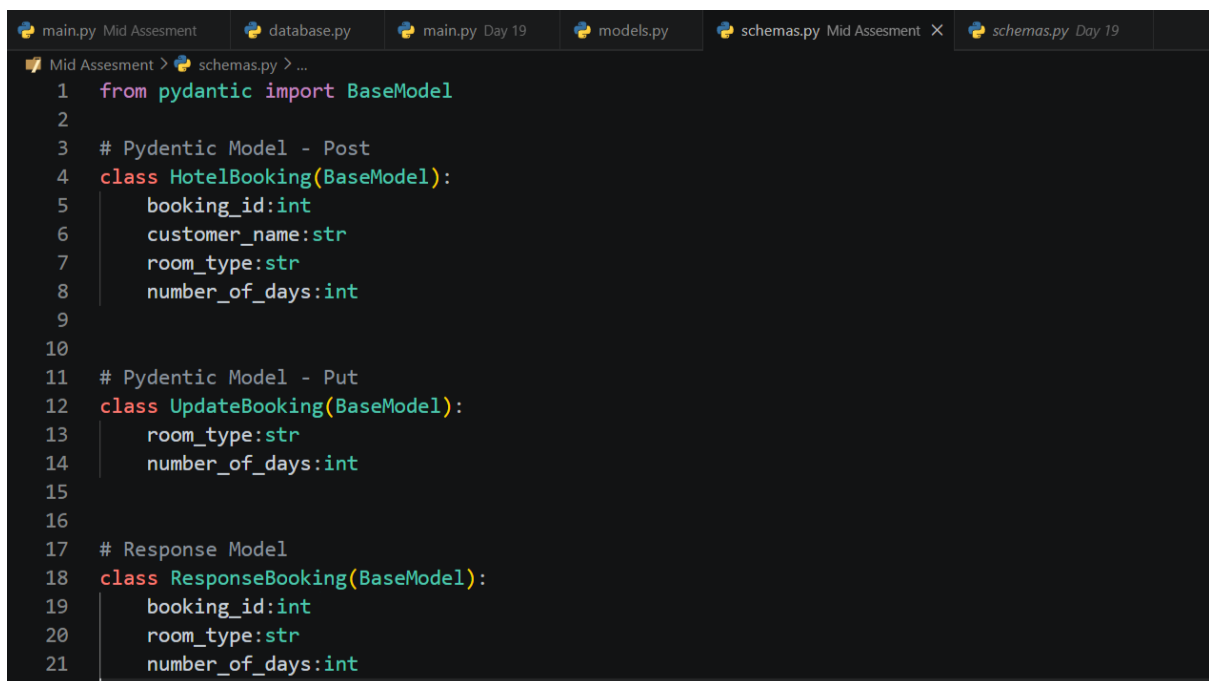


The screenshot shows a code editor with a dark theme. The top bar has tabs for 'main.py Mid Assesment', 'database.py', 'main.py Day 19', 'models.py', and 'schemas.py Mid Assesment'. The active tab is 'schemas.py Mid Assesment'. The code in the editor is as follows:

```
1 from pydantic import BaseModel
2
3 # Pydentic Model - Post
4 class HotelBooking(BaseModel):
5     booking_id:int
6     customer_name:str
7     room_type:str
8     number_of_days:int
9
10
11 # Pydentic Model - Put
12 class UpdateBooking(BaseModel):
13     room_type:str
14     number_of_days:int
```

Figure 4: Pydantic Model for Post & Put Endpoints

Task 5 - Create Response Model



The screenshot shows a code editor with a dark theme. The top bar has tabs for 'main.py Mid Assesment', 'database.py', 'main.py Day 19', 'models.py', 'schemas.py Mid Assesment', and 'schemas.py Day 19'. The active tab is 'schemas.py Mid Assesment'. The code in the editor is as follows:

```
1 from pydantic import BaseModel
2
3 # Pydentic Model - Post
4 class HotelBooking(BaseModel):
5     booking_id:int
6     customer_name:str
7     room_type:str
8     number_of_days:int
9
10
11 # Pydentic Model - Put
12 class UpdateBooking(BaseModel):
13     room_type:str
14     number_of_days:int
15
16
17 # Response Model
18 class ResponseBooking(BaseModel):
19     booking_id:int
20     room_type:str
21     number_of_days:int
```

Figure 5: Response Model

Task 6 - Create POST API

```
27
28 @app.get("/")
29 def home():
30     return {'message': "Welcome To Hotel Booking Management System ` `"}
31
32 # Add Booking
33 @app.post("/add_booking", response_model=ResponseBooking, status_code=status.HTTP_201_CREATED)
34 def add_booking(emp: HotelBookingSchema):
35     booking = HotelBooking(booking_id=emp.booking_id, customer_name=emp.customer_name, room_type=emp.
36         room_type, number_of_days=emp.number_of_days)
37
38     session.add(booking)
39     session.commit()
40     return booking
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

uvicorn - Mid Assessment

PS E:\Data Analysis\Alpha It\APIs\Mid_Assessment> uvicorn main:app --reload

Request Method: POST

Request URL: http://127.0.0.1:8000/add_booking

Request Processing Time: 0.061482906341552734

INFO: 127.0.0.1:49956 - "POST /add_booking HTTP/1.1" 201 Created

WARNING: StatReload detected changes in 'main.py'. Reloading...

INFO: Shutting down

INFO: Waiting for application shutdown.

INFO: Application shutdown complete.

INFO: Finished server process [27548]

Figure 6(a): POST API

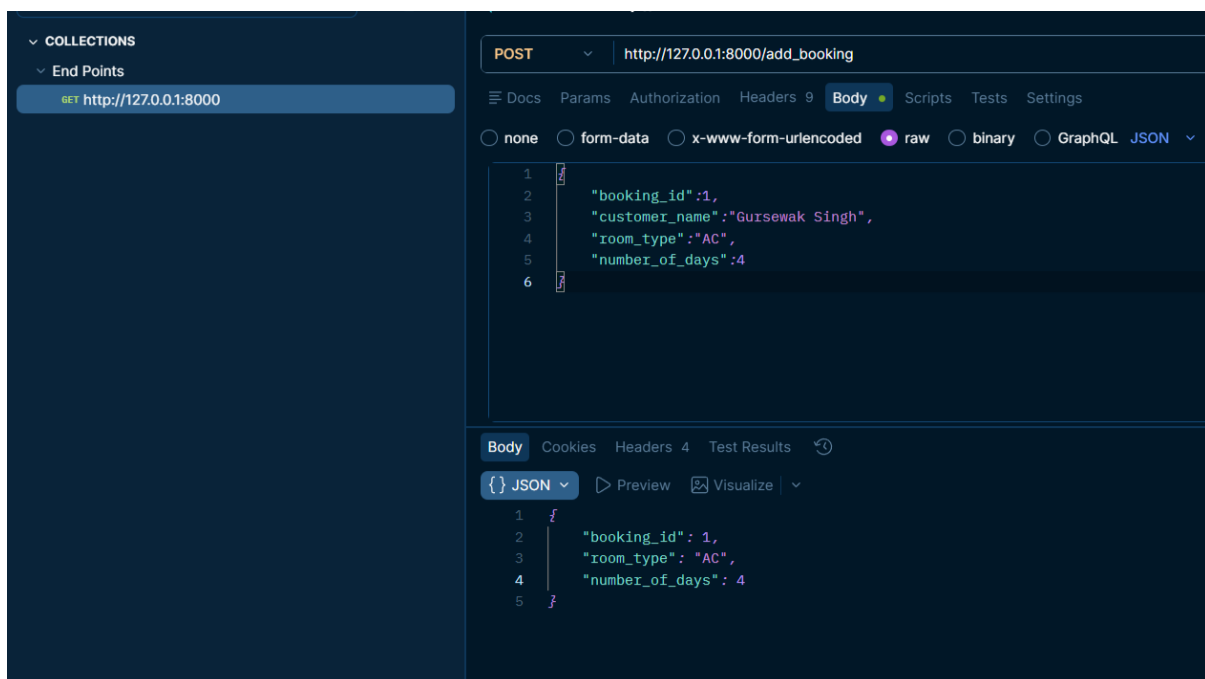


Figure 6(b): Postman Output

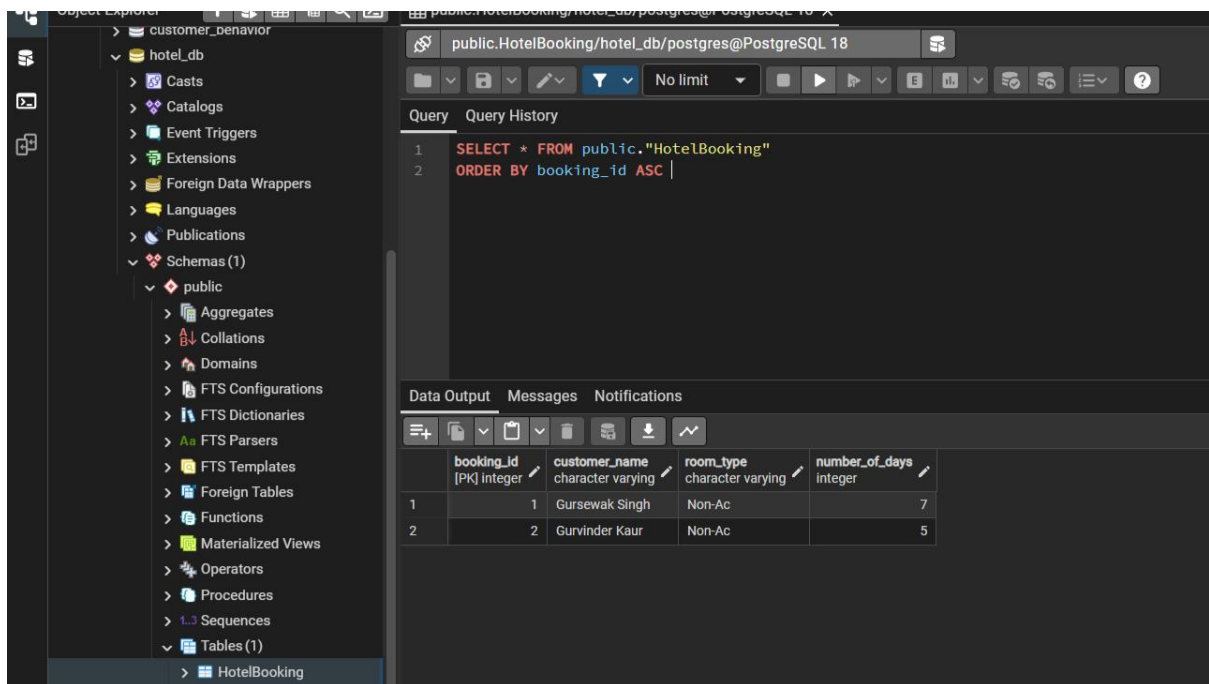


Figure 6(c): Database Output

Task 7 - Create GET API

```
41
42 # Get Booking
43 @app.get("/bookings")
44 def get_bookings():
45     bookings=session.query(HotelBooking).all()
46     #fetch data from employee table , where all means all rows
47     return bookings
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS E:\Data Analysis\Alpha It\APIS\Mid Assesment> uvicorn main:app --reload
INFO: Waiting for application startup.
INFO: Application startup complete.
Request Method: POST
Request URL: http://127.0.0.1:8000/add_booking
Request Processing Time: 0.03696751594543457
INFO: 127.0.0.1:58577 - "POST /add_booking HTTP/1.1" 201 Created
Request Method: GET
Request URL: http://127.0.0.1:8000/bookings
Request Processing Time: 0.0037238597869873047
INFO: 127.0.0.1:58579 - "GET /bookings HTTP/1.1" 200 OK

Figure 7(a): GET API

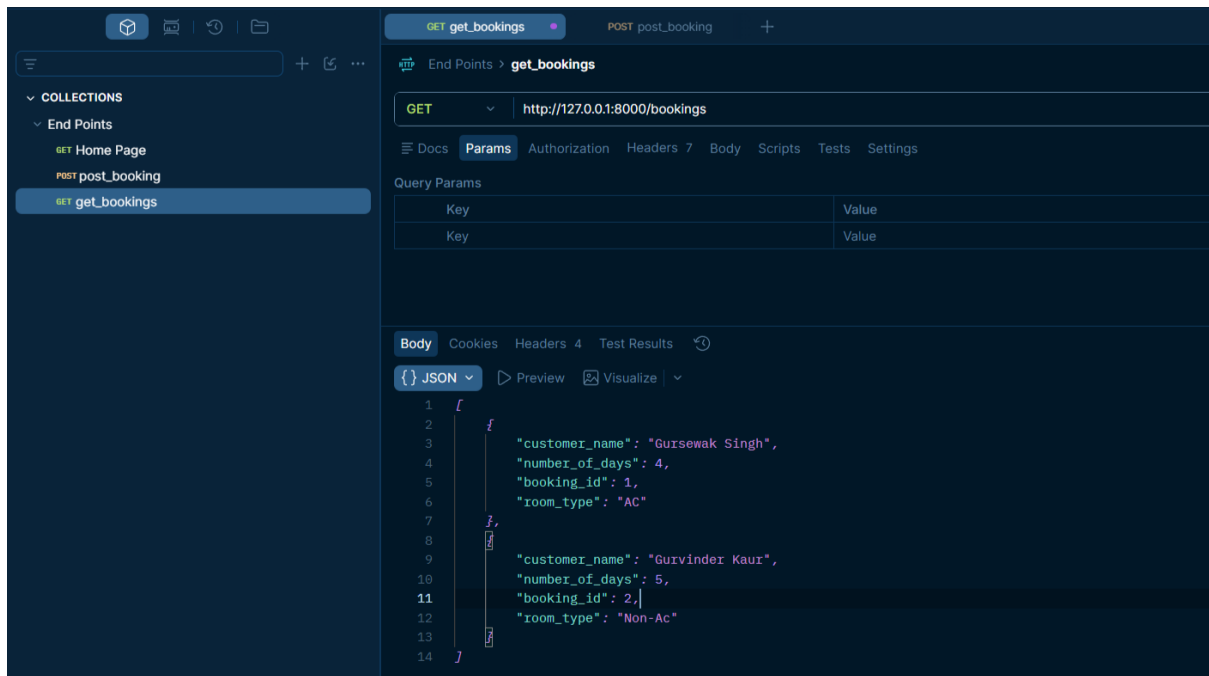
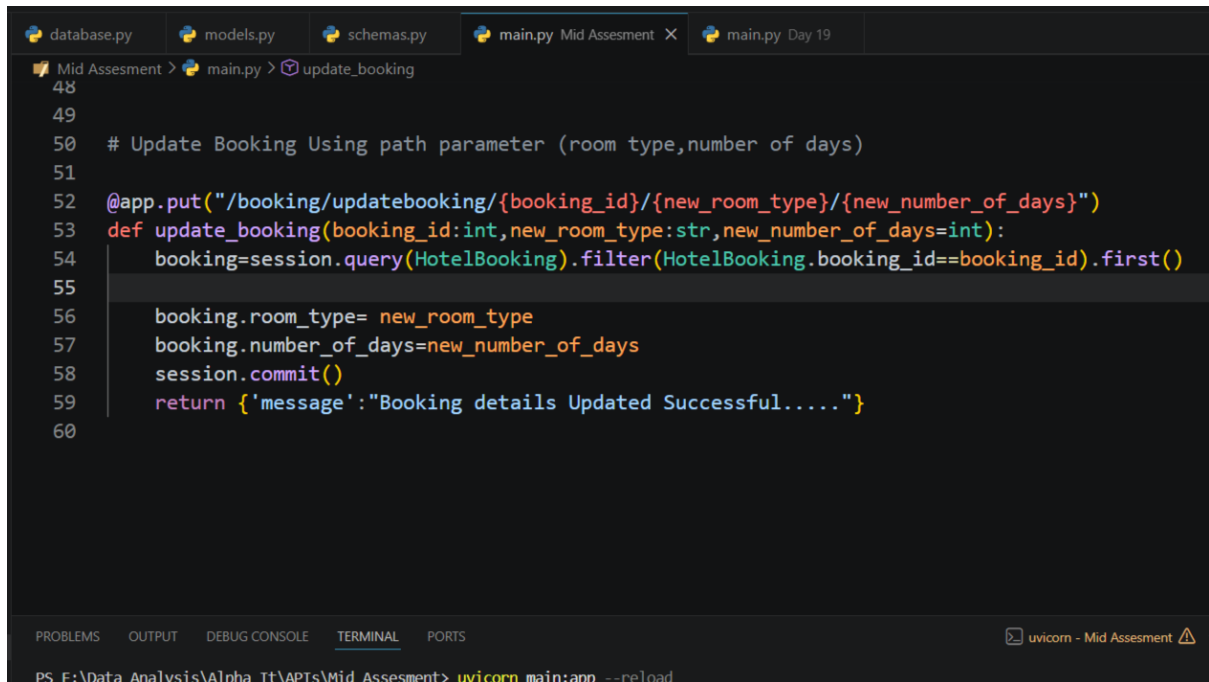


Figure 7(b): Postman Output

Task 8 - Create PUT API

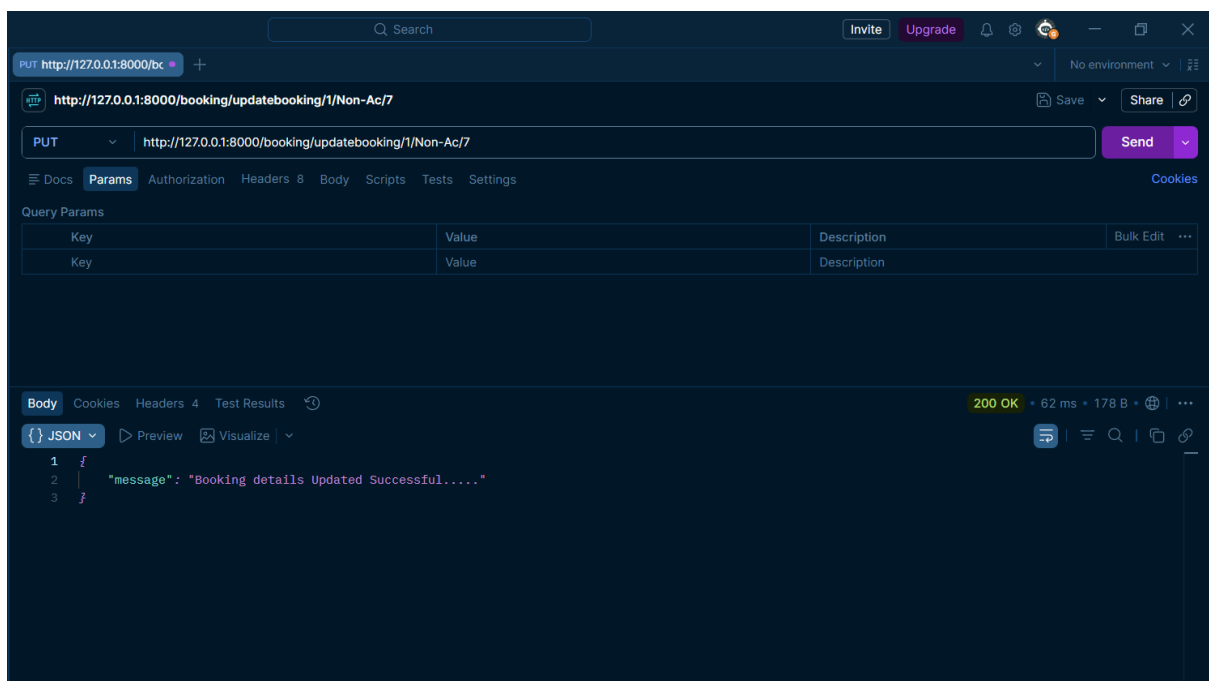


The screenshot shows a code editor with a dark theme. The top bar has tabs for 'database.py', 'models.py', 'schemas.py', 'main.py Mid Assesment', and 'main.py Day 19'. The 'main.py Mid Assesment' tab is active, showing the following Python code:

```
48
49
50 # Update Booking Using path parameter (room type,number of days)
51
52 @app.put("/booking/updatebooking/{booking_id}/{new_room_type}/{new_number_of_days}")
53 def update_booking(booking_id:int,new_room_type:str,new_number_of_days=int):
54     booking=session.query(HotelBooking).filter(HotelBooking.booking_id==booking_id).first()
55
56     booking.room_type= new_room_type
57     booking.number_of_days=new_number_of_days
58     session.commit()
59     return {'message':"Booking details Updated Successful....."}
60
```

The bottom of the editor shows a terminal with the command: `PS E:\Data Analysis\Alpha It\APIs\Mid Assesment> uvicorn main:app --reload`. The terminal output is not visible.

Figure 8(a): PUT API



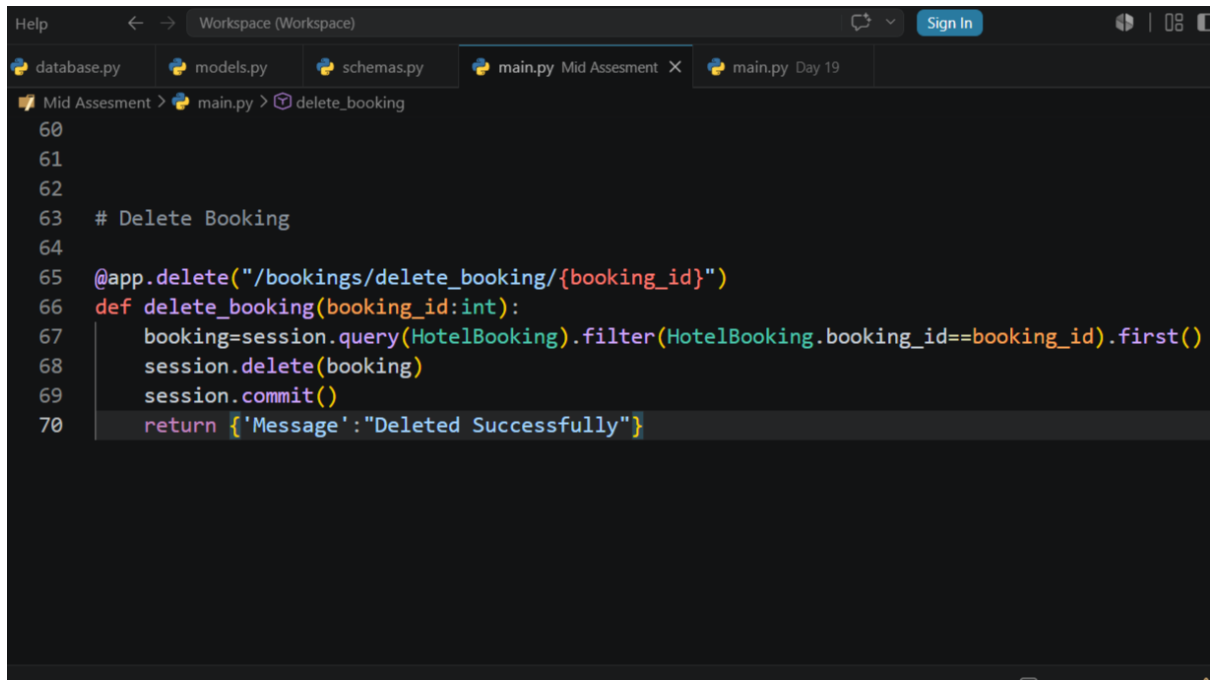
The screenshot shows the Postman interface. The top bar has a search bar and buttons for 'Invite', 'Upgrade', and a bell icon. The main area shows a PUT request to `http://127.0.0.1:8000/booking/updatebooking/1/Non-Ac/7`. The 'Params' tab is selected, showing a table with 'Key' and 'Value' columns. The 'Body' tab is selected, showing a JSON response:

```
{
  "message": "Booking details Updated Successful....."
}
```

The status bar at the bottom indicates a 200 OK response with a 62 ms response time and 178 B of data.

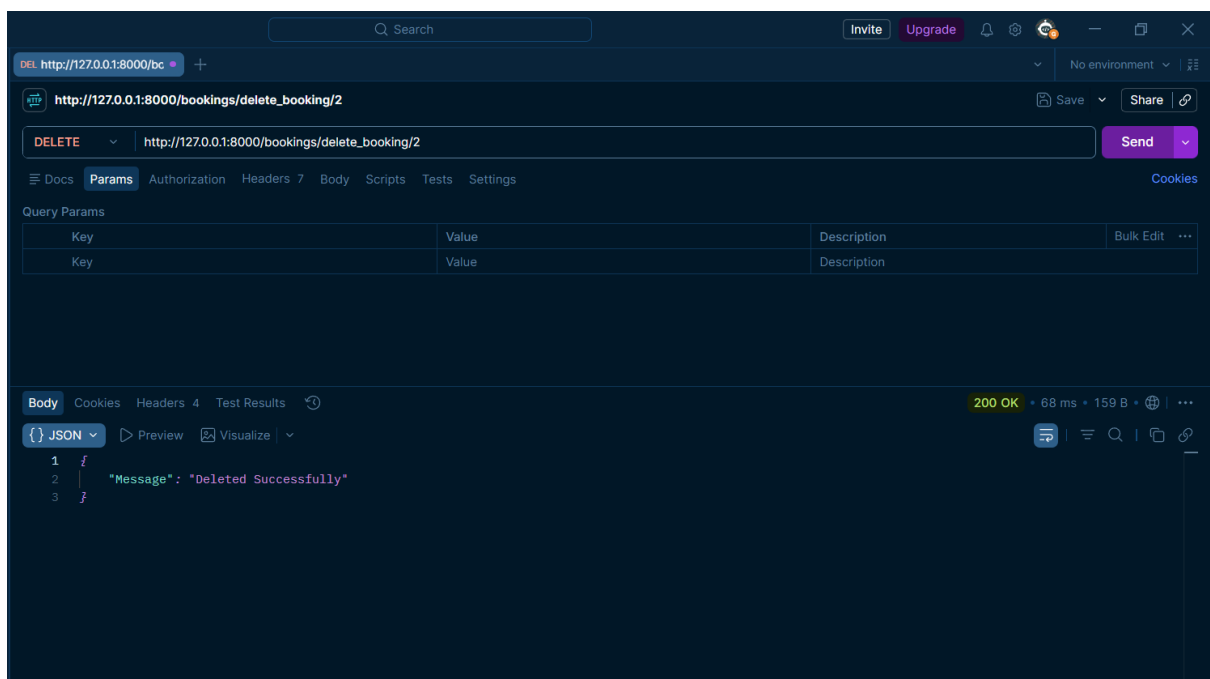
Figure 8(b): Postman Output

Task 9 - Create DELETE API



```
Help  <  >  Workspace (Workspace)  Sign In
database.py  models.py  schemas.py  main.py Mid Assesment X  main.py Day 19
Mid Assesment > main.py > delete_booking
60
61
62
63 # Delete Booking
64
65 @app.delete("/bookings/delete_booking/{booking_id}")
66 def delete_booking(booking_id:int):
67     booking=session.query(HotelBooking).filter(HotelBooking.booking_id==booking_id).first()
68     session.delete(booking)
69     session.commit()
70     return {'Message':"Deleted Successfully"}
```

Figure 9(a): PUT API



DEL http://127.0.0.1:8000/bc +

http://127.0.0.1:8000/bookings/delete_booking/2 Save Share

DELETE http://127.0.0.1:8000/bookings/delete_booking/2 Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 4 Test Results

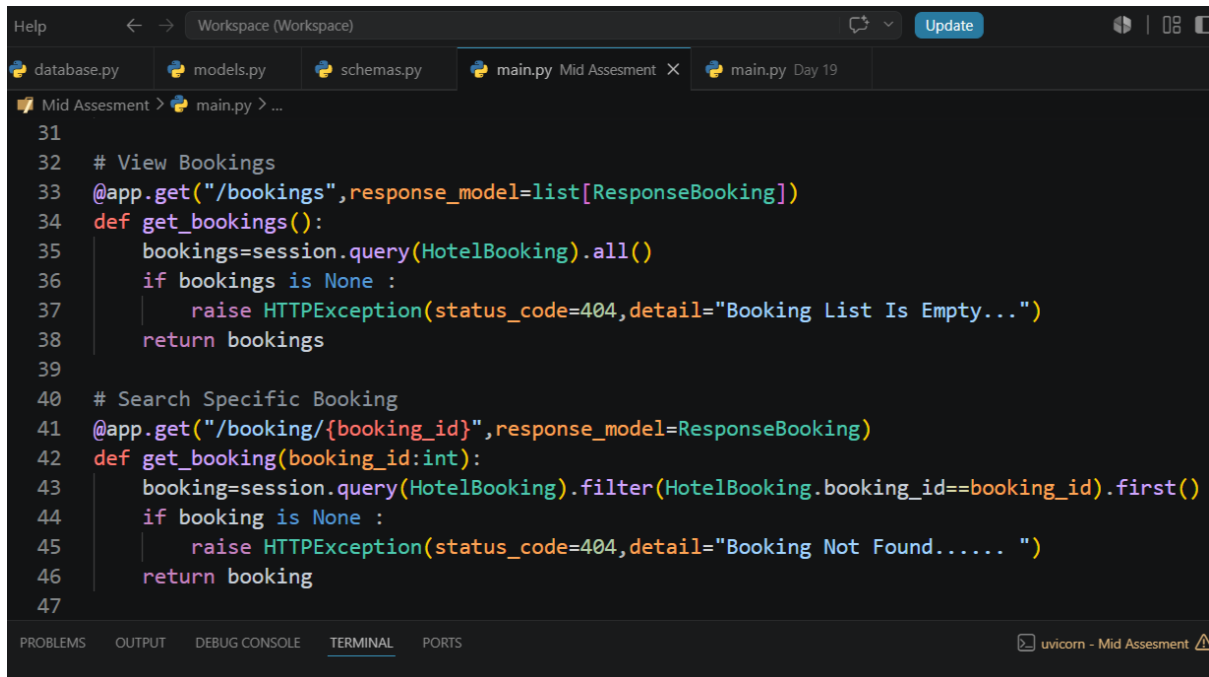
200 OK • 68 ms • 159 B •

{ } JSON Preview Visualize

```
1 {
2   "Message": "Deleted Successfully"
3 }
```

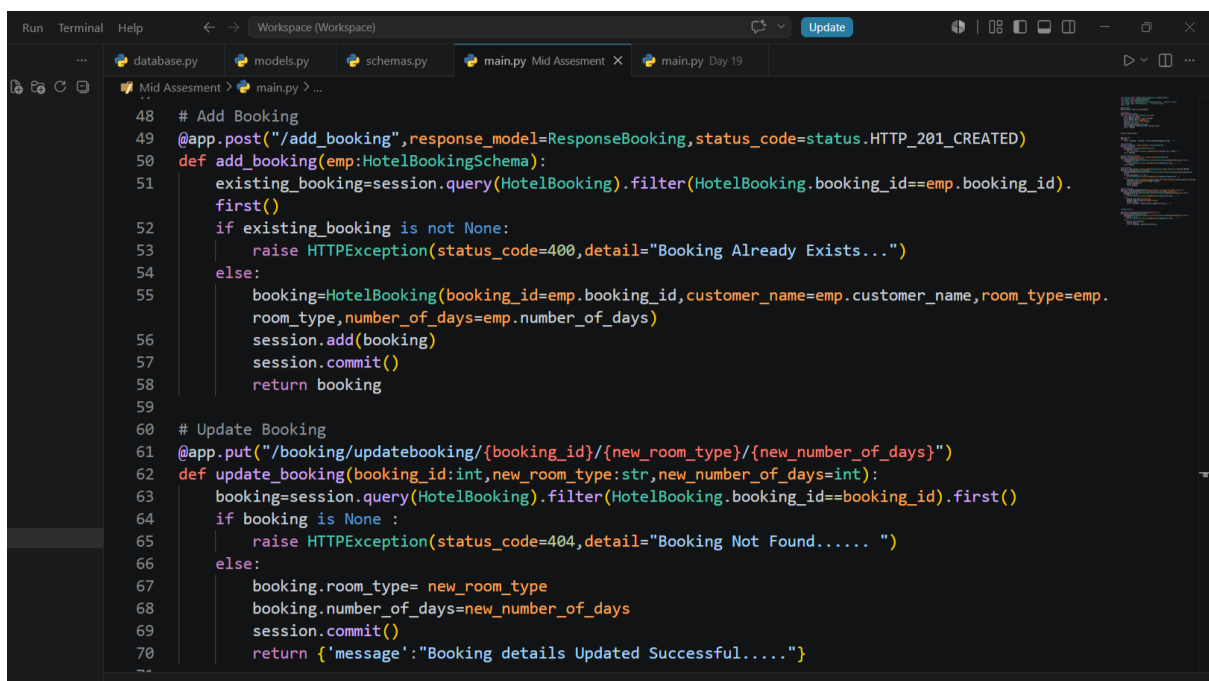
Figure 9(b): Postman Output

Task 10 - Implement Exception Handling



```
31
32 # View Bookings
33 @app.get("/bookings",response_model=list[ResponseBooking])
34 def get_bookings():
35     bookings=session.query(HotelBooking).all()
36     if bookings is None :
37         raise HTTPException(status_code=404,detail="Booking List Is Empty...")
38     return bookings
39
40 # Search Specific Booking
41 @app.get("/booking/{booking_id}",response_model=ResponseBooking)
42 def get_booking(booking_id:int):
43     booking=session.query(HotelBooking).filter(HotelBooking.booking_id==booking_id).first()
44     if booking is None :
45         raise HTTPException(status_code=404,detail="Booking Not Found..... ")
46     return booking
47
```

Figure 10(a): Implementation of Exception Handling in GET API



```
48 # Add Booking
49 @app.post("/add_booking",response_model=ResponseBooking,status_code=status.HTTP_201_CREATED)
50 def add_booking(emp:HotelBookingSchema):
51     existing_booking=session.query(HotelBooking).filter(HotelBooking.booking_id==emp.booking_id).first()
52     if existing_booking is not None:
53         raise HTTPException(status_code=400,detail="Booking Already Exists...")
54     else:
55         booking=HotelBooking(booking_id=emp.booking_id,customer_name=emp.customer_name,room_type=emp.room_type,number_of_days=emp.number_of_days)
56         session.add(booking)
57         session.commit()
58         return booking
59
60 # Update Booking
61 @app.put("/booking/updatebooking/{booking_id}/{new_room_type}/{new_number_of_days}")
62 def update_booking(booking_id:int,new_room_type:str,new_number_of_days=int):
63     booking=session.query(HotelBooking).filter(HotelBooking.booking_id==booking_id).first()
64     if booking is None :
65         raise HTTPException(status_code=404,detail="Booking Not Found..... ")
66     else:
67         booking.room_type= new_room_type
68         booking.number_of_days=new_number_of_days
69         session.commit()
70         return {'message':"Booking details Updated Successful....."}
```

Figure 10(b): Implementation of Exception Handling in POST & PUT API

```
Help  ← → Workspace (Workspace)  Update
database.py  models.py  schemas.py  main.py Mid Assesment  main.py Day 19
Mid Assesment > main.py > ...
72
73
74 # Delete Booking
75
76 @app.delete("/bookings/delete_booking/{booking_id}")
77 def delete_booking(booking_id:int):
78     booking=session.query(HotelBooking).filter(HotelBooking.booking_id==booking_id).first()
79     if booking is None :
80         raise HTTPException(status_code=404,detail="Booking Not Found..... ")
81     else:
82         session.delete(booking)
83         session.commit()
84         return {'Message':"Deleted Successfully"}
85
86
87
88
```

Figure 10(c): Implementation of Exception Handling in DELETE API

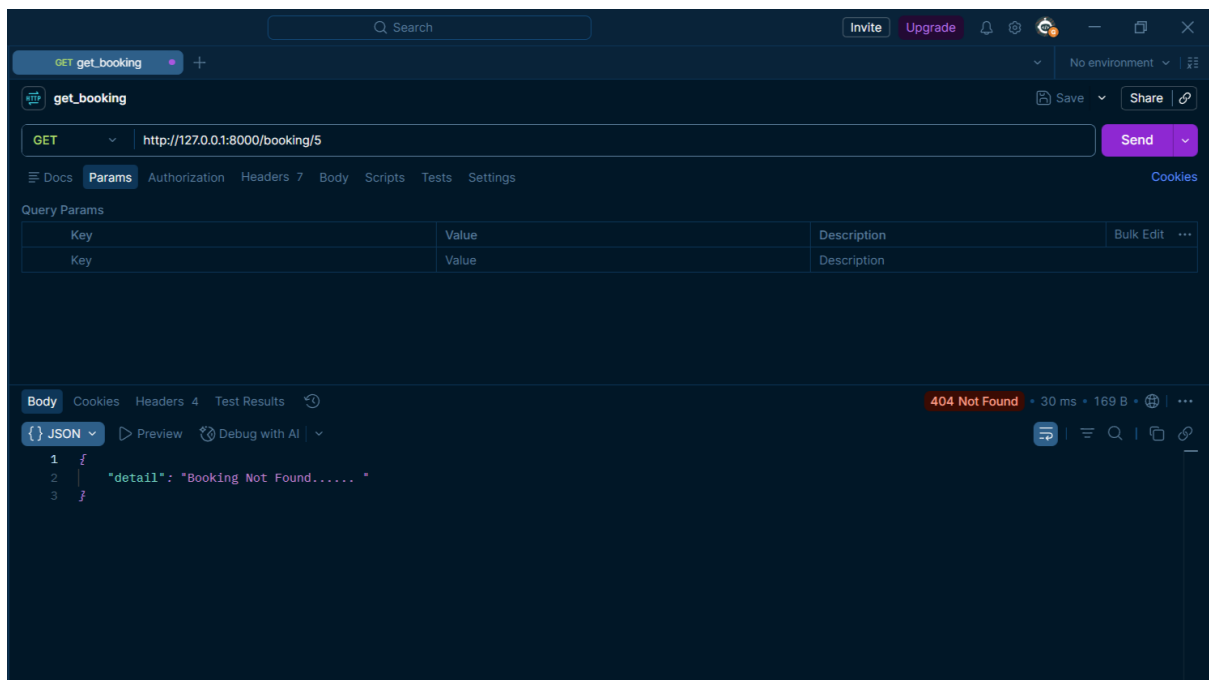
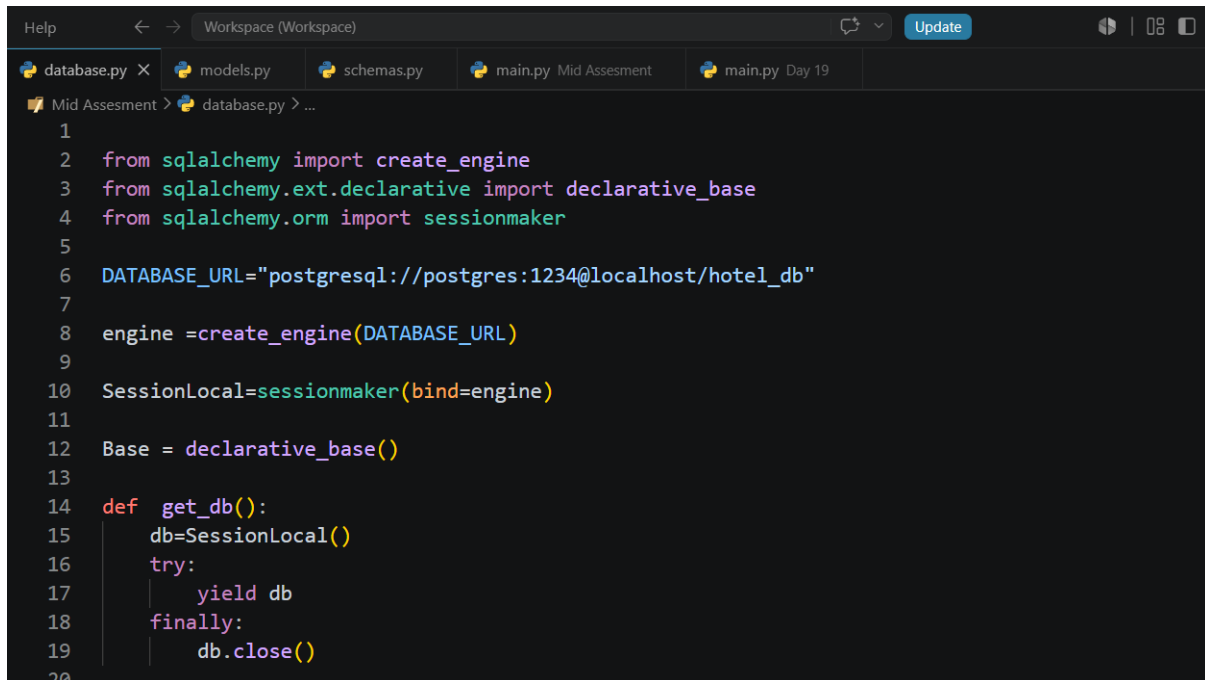


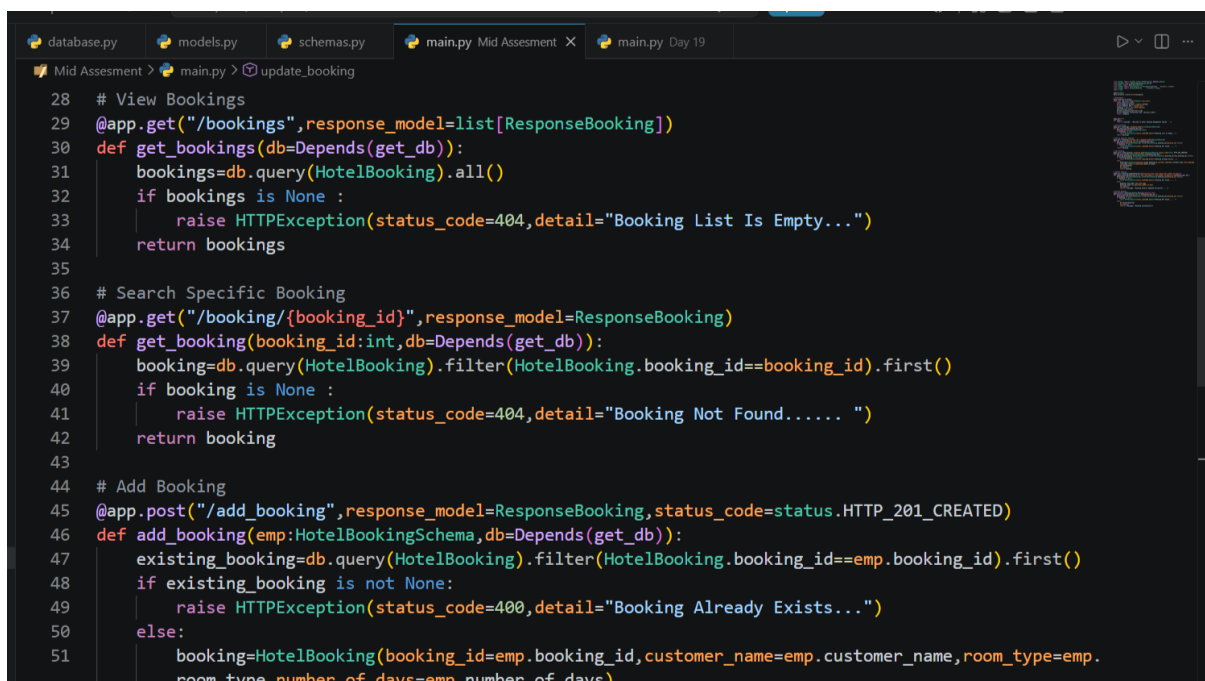
Figure 10(d): Postman Output of Exception Handling

Task 11 - Implement Dependency Injection



```
1
2 from sqlalchemy import create_engine
3 from sqlalchemy.ext.declarative import declarative_base
4 from sqlalchemy.orm import sessionmaker
5
6 DATABASE_URL="postgresql://postgres:1234@localhost/hotel_db"
7
8 engine =create_engine(DATABASE_URL)
9
10 SessionLocal=sessionmaker(bind=engine)
11
12 Base = declarative_base()
13
14 def get_db():
15     db=SessionLocal()
16     try:
17         yield db
18     finally:
19         db.close()
20
```

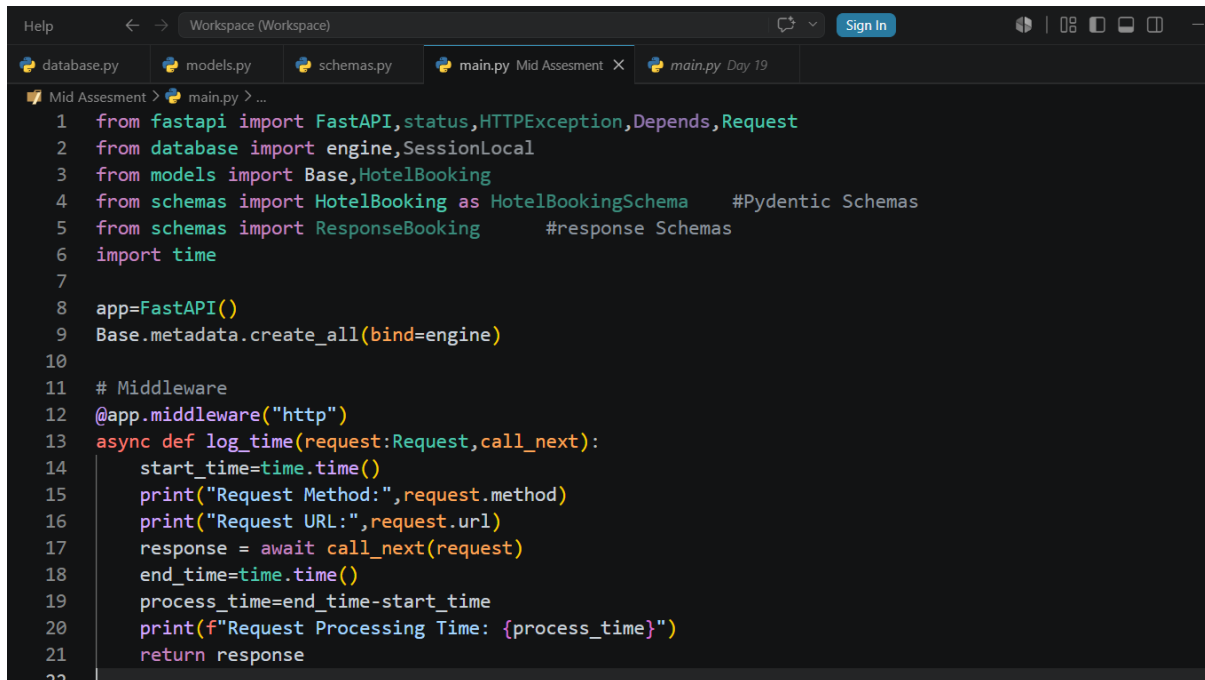
Figure 11(a): Implementation Of Dependency Injection in database.py



```
28 # View Bookings
29 @app.get("/bookings",response_model=list[ResponseBooking])
30 def get_bookings(db=Depends(get_db)):
31     bookings=db.query(HotelBooking).all()
32     if bookings is None :
33         raise HTTPException(status_code=404,detail="Booking List Is Empty...")
34     return bookings
35
36 # Search Specific Booking
37 @app.get("/booking/{booking_id}",response_model=ResponseBooking)
38 def get_booking(booking_id:int,db=Depends(get_db)):
39     booking=db.query(HotelBooking).filter(HotelBooking.booking_id==booking_id).first()
40     if booking is None :
41         raise HTTPException(status_code=404,detail="Booking Not Found..... ")
42     return booking
43
44 # Add Booking
45 @app.post("/add_booking",response_model=ResponseBooking,status_code=status.HTTP_201_CREATED)
46 def add_booking(emp:HotelBookingSchema,db=Depends(get_db)):
47     existing_booking=db.query(HotelBooking).filter(HotelBooking.booking_id==emp.booking_id).first()
48     if existing_booking is not None:
49         raise HTTPException(status_code=400,detail="Booking Already Exists...")
50     else:
51         booking=HotelBooking(booking_id=emp.booking_id,customer_name=emp.customer_name,room_type=emp.
            room_type,number_of_days=emp.number_of_days)
```

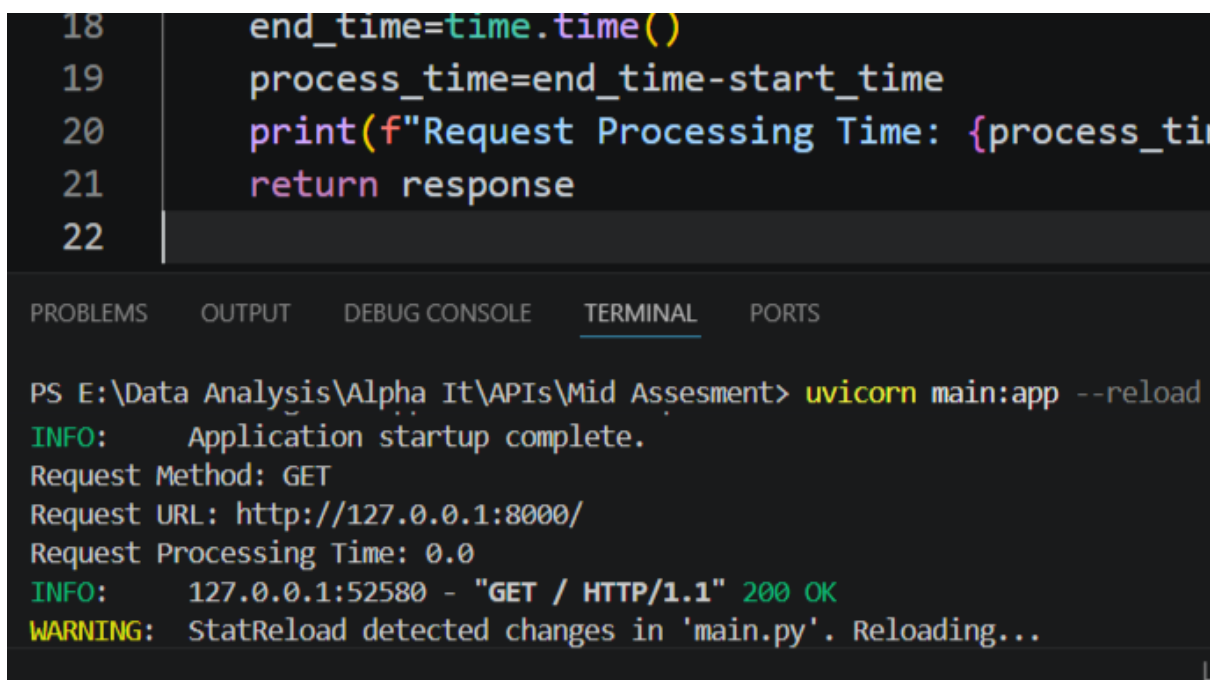
Figure 11(a): Implementation Of Dependency Injection in main.py

Task 12 - Implement Middleware



```
1 from fastapi import FastAPI, status, HTTPException, Depends, Request
2 from database import engine, SessionLocal
3 from models import Base, HotelBooking
4 from schemas import HotelBooking as HotelBookingSchema #Pydentic Schemas
5 from schemas import ResponseBooking #response Schemas
6 import time
7
8 app=FastAPI()
9 Base.metadata.create_all(bind=engine)
10
11 # Middleware
12 @app.middleware("http")
13 async def log_time(request:Request,call_next):
14     start_time=time.time()
15     print("Request Method:",request.method)
16     print("Request URL:",request.url)
17     response = await call_next(request)
18     end_time=time.time()
19     process_time=end_time-start_time
20     print(f"Request Processing Time: {process_time}")
21     return response
22
```

Figure 12(a): Implementation Of Middleware



```
18     end_time=time.time()
19     process_time=end_time-start_time
20     print(f"Request Processing Time: {process_time}")
21     return response
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\Data Analysis\Alpha It\APIs\Mid Assesment> uvicorn main:app --reload
INFO:      Application startup complete.
Request Method: GET
Request URL: http://127.0.0.1:8000/
Request Processing Time: 0.0
INFO:      127.0.0.1:52580 - "GET / HTTP/1.1" 200 OK
WARNING:   StatReload detected changes in 'main.py'. Reloading...
```

Figure 12(b): Middleware Output